

# Rapport d'Architecture et de Planification : Projet BloodLink (Approche MVP - 1,5 Mois)

**Auteur :** Manus AI  
**Date :** 17 Avril 2026  
**Projet :** BloodLink - Application de mise en relation de donneurs de sang et centres de dons  
**Équipe :** 5 étudiants, travaillant 3 jours par semaine  
**Délai :** 1,5 mois (6 semaines) pour le MVP

## 1. Introduction et Contexte

Le projet **BloodLink** vise à créer une plateforme numérique pour faciliter la gestion des dons de sang. Compte tenu d'un délai contraint de **1,5 mois (6 semaines)** pour une équipe de 5 étudiants travaillant 3 jours par semaine, une approche **MVP (Minimum Viable Product)** est indispensable. Ce rapport redéfinit l'architecture logicielle et le planning pour se concentrer sur les fonctionnalités essentielles, garantissant une première version fonctionnelle et livrable dans les délais impartis. Une phase Post-MVP sera envisagée si le temps le permet, ou comme une feuille de route pour les développements futurs.

## 2. Architecture Globale du Système (Orientée MVP)

L'architecture reste basée sur un modèle client-serveur découplé, mais la priorité est donnée à la mise en œuvre rapide des composants essentiels.

### 2.1. Vue d'Ensemble de l'Architecture

L'architecture globale reste pertinente, car elle décrit la structure fondamentale du système. Le MVP se concentrera sur l'implémentation des liens critiques entre ces composants.

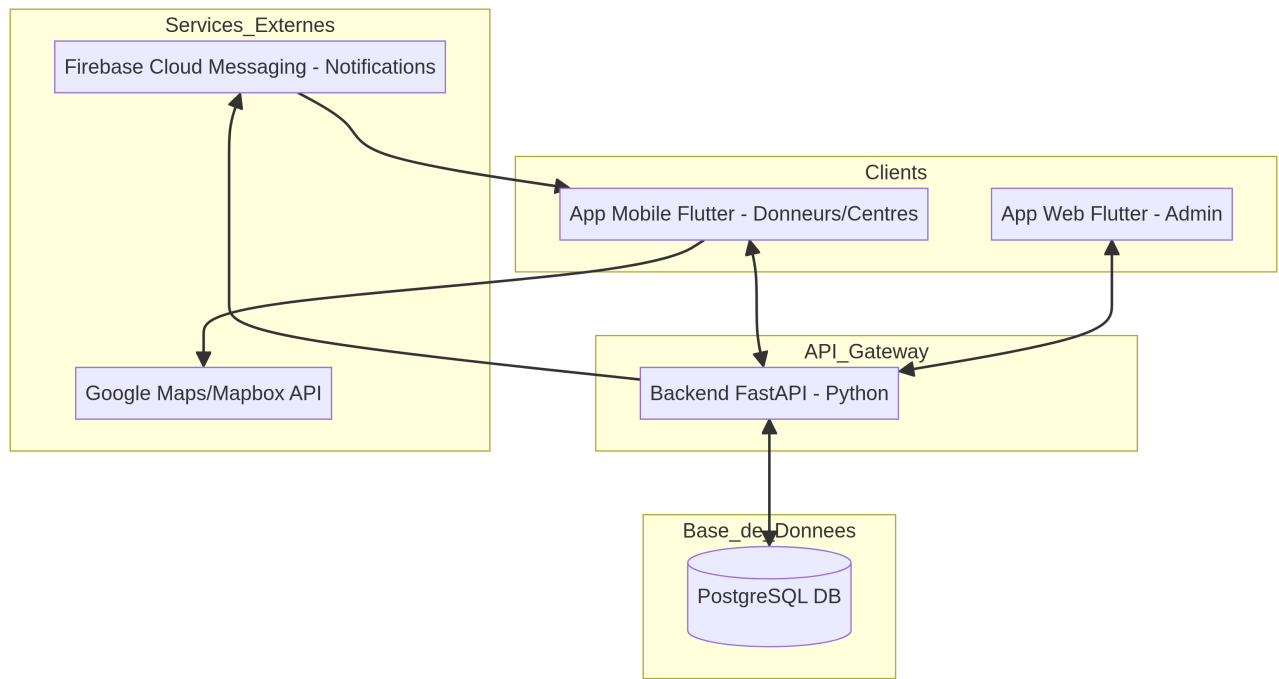


Figure 1 : Diagramme d'architecture système globale de BloodLink.

## 2.2. Choix Technologiques Approfondis (MVP Focus)

Les choix technologiques demeurent les mêmes, car ils sont efficaces et adaptés à un développement rapide et cross-platform :

- **Frontend (Mobile & Web) : Flutter**
  - **Justification MVP** : La base de code unique de Flutter est un atout majeur pour le MVP, permettant de cibler les plateformes mobile (donneurs/centres) et web (admin) simultanément avec un effort minimal. Cela accélère le développement et la livraison d'une première version fonctionnelle.
- **Backend (API REST) : FastAPI (Python)**
  - **Justification MVP** : Sa rapidité de développement, sa documentation automatique et sa performance sont cruciales pour construire rapidement les API nécessaires au MVP (authentification, profils, alertes, rendez-vous).
- **Base de Données : PostgreSQL**
  - **Justification MVP** : Sa robustesse et sa flexibilité permettent de modéliser les données essentielles du MVP et d'évoluer facilement pour les fonctionnalités futures.
- **Authentification et Sécurité : JWT (JSON Web Tokens)**
  - **Justification MVP** : JWT est suffisant pour gérer l'authentification et l'autorisation des utilisateurs (donneurs, centres, admin) dans le cadre du MVP.
- **Notifications Push : Firebase Cloud Messaging (FCM)**
  - **Justification MVP** : Essentiel pour la fonctionnalité clé des alertes d'urgence, FCM sera intégré dès le MVP.
- **Cartographie : Google Maps API ou Mapbox**
  - **Justification MVP** : La visualisation des centres et la géolocalisation pour les alertes sont des fonctionnalités critiques du MVP.

## 2.3. Principes d'Architecture Logicielle Avancés (Application MVP)

Les principes de Clean Architecture, SOLID et DDD restent des guides fondamentaux. Pour le MVP, l'accent sera mis sur :

- **Clean Architecture** : Appliquer une séparation claire des couches pour le domaine métier, l'application et l'infrastructure. Cela garantira que le code du MVP est propre et facile à étendre par la suite.
- **Principes SOLID** : Prioriser le SRP (Single Responsibility Principle) et l'OCP (Open/Closed Principle) pour les modules critiques du MVP afin d'assurer la maintenabilité et la flexibilité.
- **Domain-Driven Design** : Se concentrer sur la modélisation des entités clés du MVP (Utilisateur, Donneur, CentreSante, AlerteUrgence, RendezVous, Don) avec un langage ubique clair.

---

## 3. Modélisation de la Base de Données (PostgreSQL - MVP Focus)

---

La base de données sera conçue pour supporter les fonctionnalités du MVP, avec une attention particulière à l'évolutivité.

### 3.1. Schéma Relationnel Détaillé (MVP)

Le schéma de base de données reste le même que précédemment, car il représente les entités fondamentales du domaine. Pour le MVP, toutes ces tables seront implémentées, mais certaines colonnes ou relations moins critiques pourront être simplifiées ou omises dans un premier temps si le temps est trop contraint, puis ajoutées en Post-MVP.

| Table         | Description                       | Champs Principaux (MVP)                                                                                                                             | Relations (MVP)                           |
|---------------|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| Utilisateur   | Authentification et autorisation. | id (PK, UUID), email (UNIQUE), mot_de_passe_hash, role (ENUM), date_creation, actif.                                                                | 1:1 avec Donneur ou CentreSante           |
| Donneur       | Profil spécifique au donneur.     | utilisateur_id (PK, FK), nom, prenom, groupe_sanguin, rhesus, date_naissance, poids_kg, telephone, latitude, longitude, date_prochain_don_possible. | 1:N avec RendezVous, Don                  |
| CentreSante   | Profil spécifique au centre.      | utilisateur_id (PK, FK), nom_centre, adresse_postale, latitude, longitude, telephone.                                                               | 1:N avec AlerteUrgence, RendezVous        |
| AlerteUrgence | Alertes de besoin urgent.         | id (PK, UUID), centre_id (FK), groupe_sanguin_requis, niveau_urgence, rayon_km, date_limite, message_alerte, statut.                                | N:1 avec CentreSante                      |
| RendezVous    | Réservations de créneaux.         | id (PK, UUID), donneur_id (FK), centre_id (FK), date_heure_prevue, statut.                                                                          | N:1 avec Donneur, CentreSante             |
| Don           | Enregistrement d'un don effectif. | id (PK, UUID), donneur_id (FK), centre_id (FK), rendez_vous_id (FK, NULLABLE), date_don, valide_par_centre.                                         | N:1 avec Donneur, CentreSante, RendezVous |
| Notification  | Historique des notifications.     | id (PK, UUID), utilisateur_id (FK), titre, contenu, type_notification, date_envoi, lue.                                                             | N:1 avec Utilisateur                      |

Note : La table `HistoriquePartage` sera considérée pour la phase Post-MVP.

### 3.2. Diagramme de Classes UML

Le diagramme de classes UML ci-dessous reste une représentation fidèle des entités clés et de leurs relations, qui seront toutes implémentées pour le MVP.

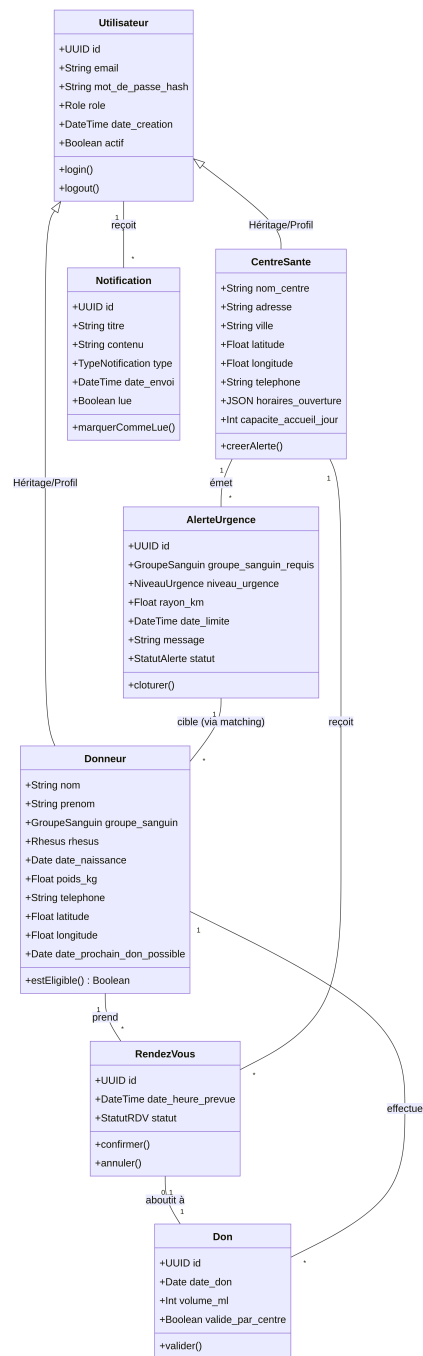


Figure 2 : Diagramme de Classes UML pour BloodLink.

### 3.3. Diagramme de Flux de Données (DFD)

Le DFD du processus de don est au cœur du MVP et sera entièrement implémenté.

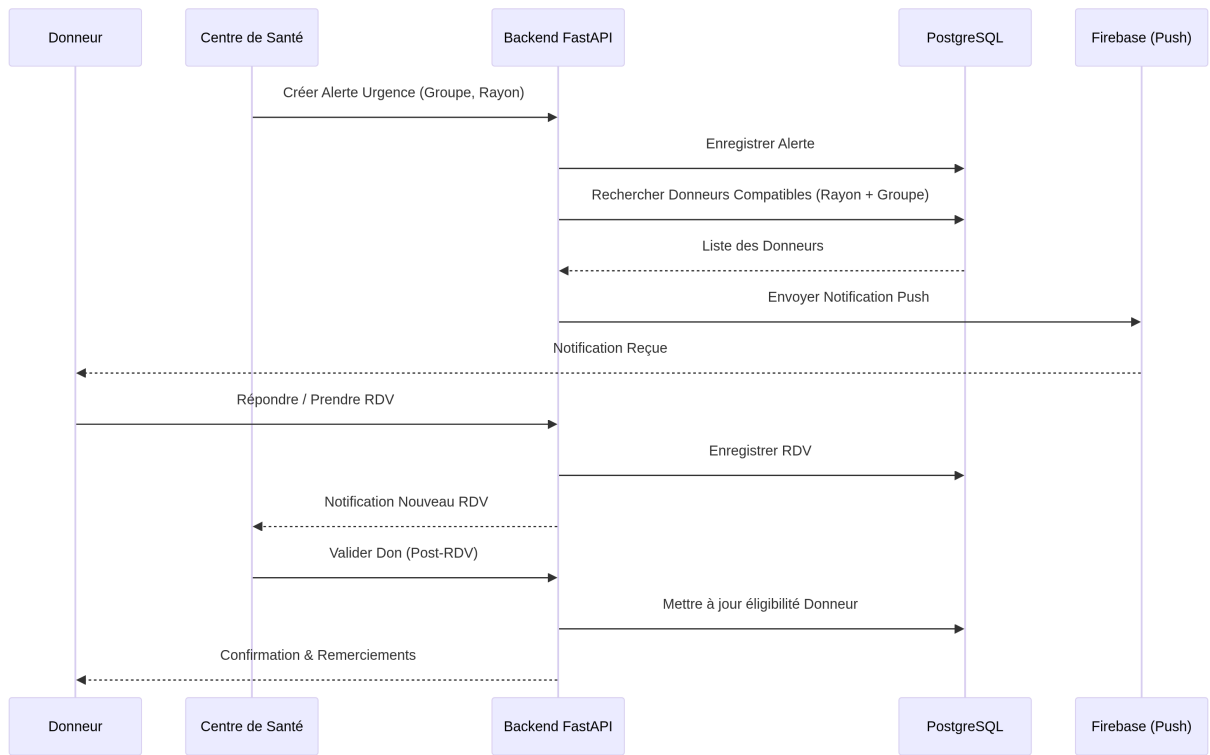


Figure 3 : Diagramme de Flux de Données pour le processus de don de sang.

3.4. Stratégies d’Optimisation de la Base de Données (MVP)

- **Indexation** : Essentielle dès le MVP pour les requêtes critiques (authentification, recherche de donneurs/centres, alertes).
- **PostGIS** : L’intégration de PostGIS sera prioritaire pour les calculs de géolocalisation nécessaires aux alertes.

4. Organisation de l’Équipe et Méthodologie (MVP Focus)

La méthodologie Scrum légère est encore plus critique pour un délai court. Chaque membre doit être polyvalent et la communication fluide.

4.1. Répartition des Rôles et Responsabilités

La répartition des rôles reste la même, mais avec une forte emphase sur la collaboration et l’aide mutuelle pour atteindre les objectifs du sprint.

| Rôle                                  | Responsabilités Clés (MVP)                                                              | Membres Suggérés |
|---------------------------------------|-----------------------------------------------------------------------------------------|------------------|
| Chef de Projet / Architecte           | Coordination, validation technique, gestion des priorités MVP, communication.           | 1 membre         |
| Développeur Backend (Lead)            | API d’authentification, profils, alertes, rendez-vous, gestion DB, sécurité.            | 1 membre         |
| Développeur Frontend (Lead)           | Architecture Flutter, gestion d’état, intégration API, UI/UX critique.                  | 1 membre         |
| Développeur Frontend (Mobile)         | UI/UX Donneur (inscription, profil, carte, RDV, alertes), intégration FCM.              | 1 membre         |
| Développeur Frontend (Web Admin) & QA | UI/UX Admin (gestion utilisateurs/centres, alertes), tests fonctionnels, documentation. | 1 membre         |

## 4.2. Méthodologie de Développement : Scrum Léger (6 Semaines)

- **Sprints** : 3 sprints de 2 semaines chacun (6 jours de travail effectif par sprint).
  - **Réunions** : Maintenir les réunions de planification, daily scrums, revues et rétrospectives. La régularité est clé.
  - **Outils** : Trello/Jira pour le suivi, Discord/Slack pour la communication.
- 

## 5. Planning Détaillé du Projet (MVP - 6 Semaines)

Ce planning est optimisé pour livrer un MVP fonctionnel en 6 semaines. Les tâches sont détaillées pour chaque membre de l'équipe.

### Phase 1 : Initialisation et Fondations (Semaines 1-2)

**Objectif** : Mettre en place l'environnement, l'architecture de base et les fonctionnalités d'authentification.

- **Semaine 1 (Jours 1-3) :**
  - **Équipe (Tous)** : Atelier de lancement, affinage des user stories MVP, configuration des outils de collaboration (Git, Trello/Jira, Discord).
  - **Backend (Lead)** : Initialisation projet FastAPI, Dockerisation, configuration PostgreSQL, ORM (SQLAlchemy), migrations (Alembic).
  - **Frontend (Lead)** : Initialisation projet Flutter (mobile/web), structure de dossiers, gestion d'état (Riverpod/BLoC), routage (GoRouter).
  - **Frontend (Mobile)** : Conception UI/UX écrans Splash, Connexion, Inscription (Figma).
  - **Frontend (Web Admin)** : Conception UI/UX Dashboard Admin (Figma).
- **Semaine 2 (Jours 4-6) :**
  - **Backend (Lead)** : Implémentation des endpoints d'authentification (inscription, connexion, JWT).
  - **Backend (Dev)** : Création des modèles de données initiaux (Utilisateur, Donneur, CentreSante) et schémas Pydantic.
  - **Frontend (Lead)** : Implémentation des écrans Splash, Connexion, Inscription en Flutter, intégration de l'API d'authentification.
  - **Frontend (Mobile)** : Développement du profil Donneur (lecture/édition des infos de base).
  - **Frontend (Web Admin)** : Développement du profil CentreSante (lecture/édition des infos de base).

### Phase 2 : Cœur de Métier - Alertes & Rendez-vous (Semaines 3-4)

**Objectif** : Implémenter les fonctionnalités clés de mise en relation et de gestion des dons.

- **Semaine 3 (Jours 7-9) :**
  - **Backend (Lead)** : Endpoints CRUD pour `AlerteUrgence` (création par centre, liste pour donneurs).
  - **Backend (Dev)** : Implémentation de la logique de matching des donneurs pour les alertes (groupe sanguin, rayon).
  - **Frontend (Lead)** : Intégration de Google Maps/Mapbox dans Flutter, affichage des centres sur la carte.
  - **Frontend (Mobile)** : Écran de liste des alertes pour le donneur, écran de création d'alerte pour le centre.
  - **Frontend (Web Admin)** : Tableau de bord Admin : liste des utilisateurs et centres.
- **Semaine 4 (Jours 10-12) :**
  - **Backend (Lead)** : Intégration du SDK Firebase Admin pour l'envoi de notifications push ciblées.
  - **Backend (Dev)** : Endpoints CRUD pour `RendezVous` (création par donneur, gestion par centre).
  - **Frontend (Lead)** : Configuration de FCM sur l'application Flutter, gestion des notifications reçues.
  - **Frontend (Mobile)** : Flux de prise de rendez-vous pour le donneur, écran d'historique des rendez-vous.

- **Frontend (Web Admin) :** Tableau de bord Admin : gestion des alertes et des rendez-vous.

### Phase 3 : Finalisation MVP & Tests (Semaines 5-6)

**Objectif :** Stabiliser le MVP, effectuer les tests et préparer le déploiement.

- **Semaine 5 (Jours 13-15) :**
    - **Équipe (Tous) :** Tests unitaires et d'intégration sur les fonctionnalités MVP.
    - **Backend (Lead) :** Endpoints pour la validation d'un don par un centre, mise à jour de l'éligibilité du donneur.
    - **Backend (Dev) :** Optimisation des requêtes de géolocalisation avec PostGIS.
    - **Frontend (Lead) :** Refactoring et optimisation du code Flutter, gestion des erreurs et des états de chargement.
    - **Frontend (Mobile) :** Écran de validation de don pour le centre, amélioration de l'UI/UX générale.
    - **Frontend (Web Admin) :** Amélioration de l'UI/UX du tableau de bord Admin, ajout de filtres/recherche.
  - **Semaine 6 (Jours 16-18) :**
    - **Équipe (Tous) :** Tests d'acceptation utilisateur (UAT) intensifs, correction des bugs critiques.
    - **Backend (Lead) :** Préparation de l'environnement de déploiement (Docker Compose pour prod, CI/CD de base).
    - **Frontend (Lead) :** Préparation des builds de production (APK/IPA pour mobile, build web pour admin).
    - **Frontend (Mobile) :** Finalisation des traductions (si nécessaire), icônes, splash screen.
    - **Frontend (Web Admin) :** Finalisation de la documentation utilisateur pour l'admin.
- 

## 6. Phase Post-MVP (Développements Futurs)

---

Si le temps le permet après le MVP, ou pour une prochaine itération, les fonctionnalités suivantes pourront être envisagées :

- **Fonctionnalités Donneur :**
    - Historique des dons détaillés avec certificats.
    - Fonction "Passer le relais" (partage social des alertes).
    - Gamification (badges, statistiques personnelles).
    - Prise de rendez-vous récurrents.
  - **Fonctionnalités Centre de Santé :**
    - Gestion des stocks de sang.
    - Statistiques avancées sur les dons et les donneurs.
    - Communication directe avec les donneurs (chat intégré).
  - **Fonctionnalités Administrateur :**
    - Gestion complète des rôles et permissions.
    - Journal d'audit détaillé.
    - Rapports personnalisables.
  - **Améliorations Techniques :**
    - Intégration de paiement (pour des services premium ou dons financiers).
    - Mise en place d'une architecture de microservices si la complexité augmente.
    - Amélioration de la résilience et de la scalabilité de l'infrastructure.
-

## 7. Recommandations Supplémentaires et Outils

---

### 7.1. Environnements de Développement Intégrés (IDE)

Les mêmes IDE sont recommandés, leur efficacité étant encore plus cruciale en mode MVP :

- **Pour Flutter (Mobile & Web) :** Visual Studio Code (VS Code) ou Android Studio / IntelliJ IDEA.
- **Pour FastAPI (Backend Python) :** Visual Studio Code (VS Code) ou PyCharm.

### 7.2. Intégration de l'Intelligence Artificielle (IA) dans le Flux de Travail (MVP)

L'IA peut être un accélérateur significatif même pour le MVP :

- **Assistants de Code (GitHub Copilot, Tabnine) :** Utiliser ces outils pour générer rapidement des ébauches de code, des fonctions utilitaires et des tests, réduisant le temps de développement.
- **Génération de Tests :** Utiliser l'IA pour générer des tests unitaires pour les composants critiques du MVP (authentification, matching d'alertes).
- **Documentation Automatisée :** L'IA peut aider à générer des docstrings et des commentaires pour le code, assurant une documentation minimale mais essentielle pour le MVP.

### 7.3. Outils de Collaboration et de Gestion de Projet

- **Gestion de Version : Git et GitHub/GitLab :** Indispensable pour la collaboration rapide et le suivi des modifications.
  - **Gestion de Projet : Trello/Jira :** Pour une gestion agile des tâches et une visibilité claire de l'avancement du MVP.
  - **Communication : Discord/Slack :** Pour une communication instantanée et la résolution rapide des blocages.
  - **Conception UI/UX : Figma/Adobe XD :** Pour des maquettes rapides et itératives des interfaces du MVP.
- 

## 8. Conclusion

---

Ce rapport a redéfini l'architecture et le planning du projet BloodLink pour une approche MVP sur 1,5 mois. En se concentrant sur les fonctionnalités essentielles, en s'appuyant sur des technologies performantes et des principes de génie logiciel solides, et en exploitant les outils d'IA, votre équipe est équipée pour livrer une première version fonctionnelle et impactante. La discipline, la communication et la flexibilité seront vos meilleurs atouts pour relever ce défi.

---

## 9. Références

---

[1] Flutter. (n.d.). *Build apps for any screen*. Consulté le 17 avril 2026, de <https://flutter.dev/> [2] FastAPI. (n.d.). *The modern, fast (high-performance) web framework for building APIs with Python 3.7+ based on standard Python type hints*. Consulté le 17 avril 2026, de <https://fastapi.tiangolo.com/> [3] PostgreSQL. (n.d.). *The world's most advanced open source relational database*. Consulté le 17 avril 2026, de <https://www.postgresql.org/> [4] Firebase. (n.d.). *Firebase Cloud Messaging*. Consulté le 17 avril 2026, de <https://firebase.google.com/docs/cloud-messaging> [5] Martin, R. C. (2012). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall. [6] Martin, R. C. (2000). *Design Principles and Design Patterns*. Consulté le 17 avril 2026, de [https://web.archive.org/web/20150906143604/http://www.objectmentor.com/resources/articles/Principles\\_and\\_Patterns.pdf](https://web.archive.org/web/20150906143604/http://www.objectmentor.com/resources/articles/Principles_and_Patterns.pdf) [7] Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.

*Fin du rapport.*